



LABORATORY MANUAL

SC4172
IoT - TinyML

Lab Experiment #4

Gender detection via voice

COLLEGE OF COMPUTING AND DATA SCIENCE
NANYANG TECHNOLOGICAL UNIVERSITY

1. **OBJECTIVES**

To train a CNN model to detect the gender of a person via voice. MFCC features will be extracted from the Google Common Voice Dataset to be used as input data to the CNN model. The resultant model is converted to TFLite model and downloaded to nano33 sense board.

2. **LABORATORY**

This experiment is conducted at the **Software Laboratory 1 (Location: N4-01a-02)**.

3. **HARDWARE EQUIPMENT**

- Computer installed with Anaconda and Tools for TensorFlow ML.
- Arduino Nano 33 BLE Sense board
- Arduino IDE installed with TensorFlow Lite (TFLite) library

4. **REFERENCES**

- TinyML book by Pete Warden, Daniel Situnayake. 2019.
<https://www.oreilly.com/library/view/tinyml/9781492052036/>
- Nano33 BLE Sense Board. <https://docs.arduino.cc/hardware/nano-33-ble-sense/>
- SC4172 Lecture Notes
- This lab references a course project in the past. The student report describing the steps taken can be found in the Appendix.
- Common Voice Dataset in Kaggle. <https://www.kaggle.com/>

5. **INTRODUCTION**

MFCC (Mel-frequency cepstral coefficients) is commonly used as input feature data to train speech and audio related neural network models as it could better represent human voice and audio characteristics. This lab implements a gender detection application by training a CNN model using Common Voice Dataset from Kaggle. Kaggle, subsidiary of Google, is an online platform for data science and machine learning that offers tools, datasets, and a collaborative environment for users.

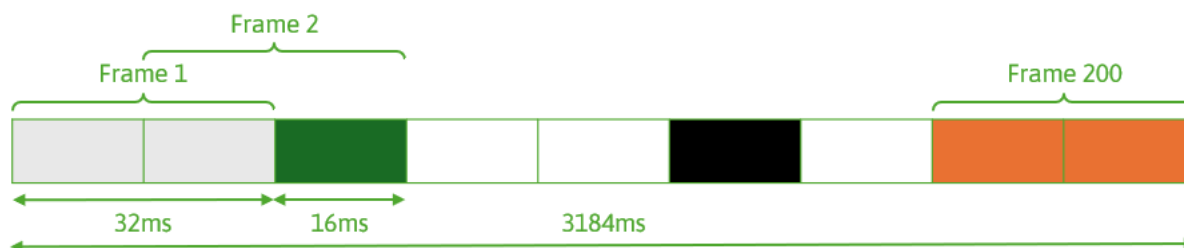
Download lab related zip files from NTULearn

- Unzip the project zip file into folder and rename the folder to “project_gender”.
- Unzip the data zip file to a folder named data.
- ‘data’ folder and ‘project_gender’ folder are at the same level in folder hierarchy.

Some of the key files and folders in the zip folder are:

- **project_gender.ipynb.**
 - Jupyter notebook to train the gender voice model based on Common Voice Dataset and MFCC.
- **audio_gender_classification_model.h.**
 - TFLite header file generated by Jupyter notebook. To be used in Arduino Sketch IDE.

- **project_gender.ino**
 - Arduino (IDE 2.x) file to generate executable for downloading to nano33 Sense board.
- **cv-valid-dev.csv, cv-valid-test.csv, cv-valid-training.csv.**
 - csv files adapted from Common Voice site in Kaggle. Contains information of speech audio file. This project combined the three datasets to one combined dataset (combined_dataset_gender) before splitting it to its own version of training, validation and test dataset. See code for more details.
 - Note that the csv files in the zip file are a subset of the original csv files in Kaggle. This is to reduce the time needed to train the dataset during lab session. This leads to lower accuracy, so students are strongly encouraged to attempt the training with the full dataset to see the optimal results. The performance result in the appendix is wrt the full Common Voice dataset.
- **data folder**
 - Contain speech files (mp3 format) in cv-valid-dev, cv-valid-test, cv-valid-training sub folders. This will be used to create the combined_dataset_gender sub folder.
- Input Data Frame Format and MFCC parameters
 - **N_MFCC = 13:** First 13 coefficients contain most speech-relevant information
 - **N_MELS = 26:** Number of Mel filter banks
 - **MIN_FREQ = 50:** Low frequency limit
 - **MAX_FREQ = SAMPLING_RATE / 2:** Nyquist frequency
 - **SAMPLING_RATE = 16000:** Common sampling rate of 16kHz
 - **N_FRAMES = 200:** Number of frames per audio sample
 - **FRAME_SIZE = 512:** 32ms window at 16kHz
 - **HOP_LENGTH = 256:** 16ms time step, 50% overlap



6. Code Execution

Refer to the report in the appendix for details on the Jupyter Notebook.

- Ensure the code can be executed in the platform of choice (Anaconda, Google Colab etc), and the resultant model header file can be compiled with the project_gender.ino file in the Arduino IDE.
- As mentioned earlier, the current model will not be very accurate as only a small subset of the common voice dataset is used. The small dataset allows you to verify the code in a manageable period within the lab session. You will notice the accuracy drop further with real-time voice input data captured from nano33 on board microphones due to noise and quality of captured voice data. Try training with more data from the original common voice dataset.
- **Suggested investigation:**

- Analyze the input-data management (frame size, buffering scheme, N_FRAMES, FRAME_SIZE, and HOP_LENGTH etc) used in the Arduino Sketch. Test with different schemes to see if that would result in different performance. Bear in mind that some settings may result in insufficient memory error as the memory on nano33 sense is limited.
- The model was not trained on ambient noise or silence, so it attempts to classify input even during pauses in speech or in the absence of a speaker, leading to unreliable predictions in those cases. Try to include ambient noise in the training dataset to resolve this.
- Benchmark the Arduino Code to see which functions/stages e.g. data input, MFCC extraction, model prediction etc, resulted in the most delay, and quantify the time delay. Doing so allows you to identify bottlenecks in the processing flow so appropriate trade-off may be applied between accuracy and response time.

Appendix – Course Project Report on Gender Voice Detection via MFCC

Gender Classification based on Voice

1. Aim

The goal of this project is to train a CNN model to classify a speaker's gender based on their voice. For this project, gender is considered in the binary sense as "female" or "male" based on stereotypical characteristics. MFCCs are extracted from the audio dataset and used as input features for training the CNN model. The model then outputs a confidence score for binary classification.

A potential use case for this project is assisting transgender individuals undergoing voice-affirming surgery/training by providing real-time feedback on how closely their voice aligns with their target gender presentation. The voice gender classifier can thus serve as a lightweight, accessible tool for tracking vocal changes over time.

2. Data Preparation and Processing

2.1 Dataset used

The dataset used is the Common Voice dataset on Kaggle, which is a collection of audio files of text data spoken by people of various demographics.

This dataset was chosen as it had a sufficient number of data points for training (>50k) with properly labelled 'gender' classes and the audio files were clean and clear.

2.2 Selecting and setting the MFCC parameters

The following parameters were set with these values:

- **N_MFCC = 13**
 - Number of MFCC coefficients to retain
 - Standard choice of 13, as the first 13 coefficients contain most speech-relevant information
- **N_MELS = 26**
 - Number of Mel filter banks
 - Common number of 26
- **N_FRAMES = 200**
 - Number of frames per audio sample
 - 200 to accommodate the smaller RAM on the Arduino
 - This number was decided upon through trial and error on the Arduino, to minimize the amount of space required by the audio buffer while having sufficient data for the model input.
- **SAMPLING_RATE = 16000**
 - Common sampling rate of 16kHz for speech processing
- **FRAME_SIZE = 512**
 - 512 corresponds to a 32ms window at 16kHz
- **HOP_LENGTH = 256**
 - 256 corresponds to a 16ms step at 16kHz
 - 50% overlap between successive frames
- **MIN_FREQ = 50**
 - Low-frequency limit
 - Set to 50Hz
- **MAX_FREQ = SAMPLING_RATE / 2**
 - High-frequency limit
 - Set to 8000Hz which is the Nyquist frequency for 16kHz ($16000\text{Hz} / 2 = 8000\text{Hz}$)

2.3 Extracting data to be used

The dataset was cleaned to only include the data entries where 'gender' was not null, and only the 'female' and 'male' labels were kept since the 'other' label had insufficient entries to be significant. This yielded a total of 57301 'male' entries and 19035 'female' entries.

As each audio file had a length of 4-10s and the audio length required for input is only 3184ms (32ms + (16ms * 199)), audio snippets were first extracted from each audio file such that multiple snippets could be derived from each audio file, to fully utilize the data available. This yielded a total of 98065 'male' audio snippets and 34383 'female' audio snippets.

The audio snippets were then normalized to range [-1,1] and MFCCs of the audio snippets were calculated using the Python 'librosa' library.

```
def normalize_audio(audio):
    # Perform normalization to range [-1, 1]
    max = np.max(np.abs(audio))
    if max > 0:
        audio /= max
    return audio

# Function to get array from audio file
def get_audio_snippets_from_audio_file(audio_file):
    y, sr = librosa.load(audio_file, sr=SAMPLING_RATE)

    target_length = FRAME_SIZE + (N_FRAMES - 1) * HOP_LENGTH

    audio_array = []

    if len(y) < target_length:
        # Pad the audio array if it is shorter than the target length
        padding_size = target_length - len(y)
        pad_left = padding_size // 2
        pad_right = padding_size - pad_left
        y = np.pad(y, (pad_left, pad_right)) # Pad both sides

    audio_array.append(y)
    else:
        num_audio_snippets = len(y) // target_length

        for i in range(num_audio_snippets):
            audio_snippet = y[i * target_length:(i + 1) * target_length]
            audio_array.append(audio_snippet)

        # If the remainder of the audio doesn't fill a full snippet
        remainder = len(y) % target_length
        if remainder > 0:
            # Pad the last snippet to reach the target length
            audio_snippet = y[-remainder:] # Get the last part
            pad_left = (target_length - remainder) // 2
            pad_right = (target_length - remainder) - pad_left
            audio_snippet = np.pad(audio_snippet, (pad_left, pad_right))
            audio_array.append(audio_snippet)

    # Convert to numpy array
    audio_array = np.array(audio_array)

    return audio_array

# Function to get MFCC from array
def extract_mfcc(data):
    # Compute the MFCC
    mfcc = librosa.feature.mfcc(
        y=data, # audio time series
        sr=SAMPLING_RATE, # sampling rate of y
        n_mfcc=N_MFCC, # number of MFCCs to return
        n_mels=N_MELS, # number of Mel bands to generate
        n_fft=FRAME_SIZE, # length of the FFT window
        hop_length=HOP_LENGTH, # number of samples between successive frames
        fmin=MIN_FREQ, # lowest frequency (in Hz)
        fmax=MAX_FREQ, # highest frequency (in Hz)
        window='hann',
        center=False, # disable centering of the frames
        dct_type=2, # discrete cosine transform (DCT) type
        norm='ortho', # setting norm='ortho' uses an ortho-normal DCT basis
        power=2, # exponent applied to the spectrum
    )

    return mfcc

# Function to extract MFCCs for a given dataframe
def extract_mfccs_for_dataset(df, audio_folder):
    mfccs = []
    labels = []

    for index, row in df.iterrows():
        filename_in_df = row['filename']
        filename_in_dataset = filename_in_df.replace("/", "_")
        gender = row['gender']

        # Path to audio file (update this path according to your setup)
        audio_path = os.path.join(audio_folder, filename_in_dataset)

        snippets = get_audio_snippets_from_audio_file(audio_path)

        for snippet in snippets:
            normalized_snippet = normalize_audio(snippet)
            mfcc = extract_mfcc(normalized_snippet)
            mfccs.append(mfcc)
            labels.append(gender)

    # Convert to numpy arrays
    mfccs = np.array(mfccs)
    labels = np.array(labels)

    return mfccs, labels
```

As there are significantly more 'male' data points than 'female' data points, some 'male' data points were randomly dropped to achieve an equal number of 'male' and 'female' data points. This is to ensure that the model does not develop a bias toward the majority class, which could lead to inaccurate predictions and poor generalization to unseen data. This yielded 34383 data points for each class.

The dataset was then split into a train, validation and test ratio of 70:15:15 for training purposes. This yielded 48136 data points for the train dataset, 10315 data points for the validation dataset and 10315 data points for the test dataset.

3. Model Training

3.1 Model architecture

The model was trained using a CNN architecture, with 2 Conv2D layers to capture more low-level features from the spectrograms, followed by a DepthwiseConv2D (instead of a Conv2D) to ensure the model does not have too many parameters. The ReLU activation function was used for each layer. A MaxPooling2D layer was used after each convolution layer.

A GlobalAveragePooling2D layer was used after, followed by a Dense layer and finally a Dense layer of 1 neuron for the output neuron.

The Adam optimizer was used and early stopping and a learning rate scheduler was also employed. Dropout layers were employed for the second Conv2D layer and the first Dense layer to prevent the model from overfitting.

This yielded a total number of 6209 trainable parameters, keeping the model lightweight for deployment on the Arduino.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 13, 200, 16)	160
max_pooling2d (MaxPooling2D)	(None, 6, 100, 16)	0
conv2d_1 (Conv2D)	(None, 6, 100, 32)	4,640
dropout (Dropout)	(None, 6, 100, 32)	0
max_pooling2d_1 (MaxPooling2D)	(None, 3, 50, 32)	0
depthwise_conv2d (DepthwiseConv2D)	(None, 3, 50, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 1, 25, 32)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 32)	0
dense (Dense)	(None, 32)	1,056
dropout_1 (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 1)	33

Total params: 6,209 (24.25 KB)

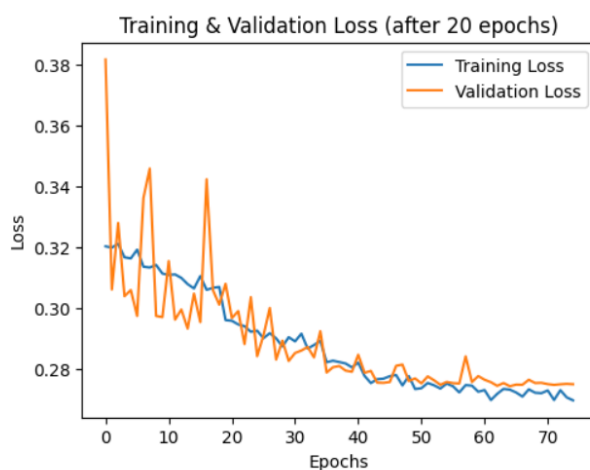
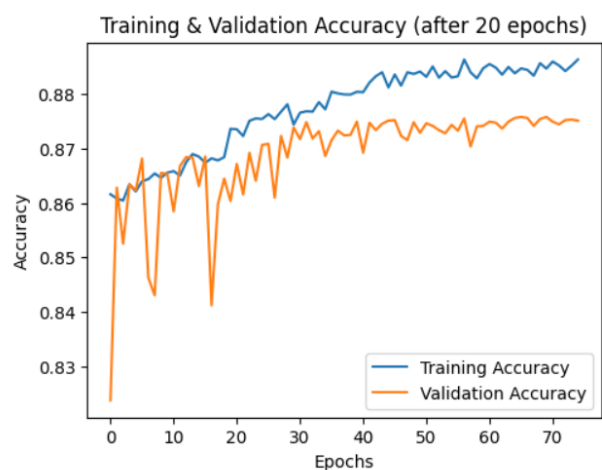
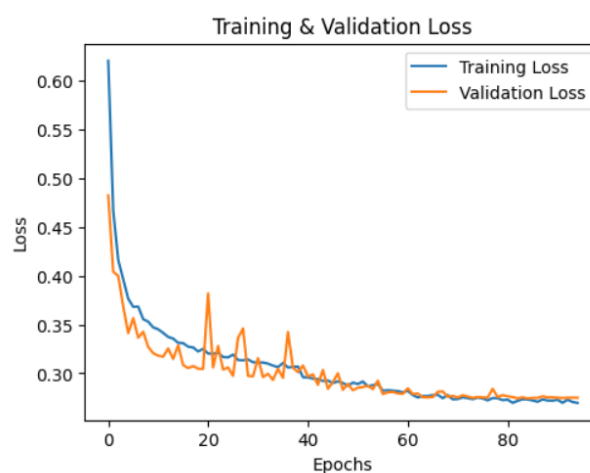
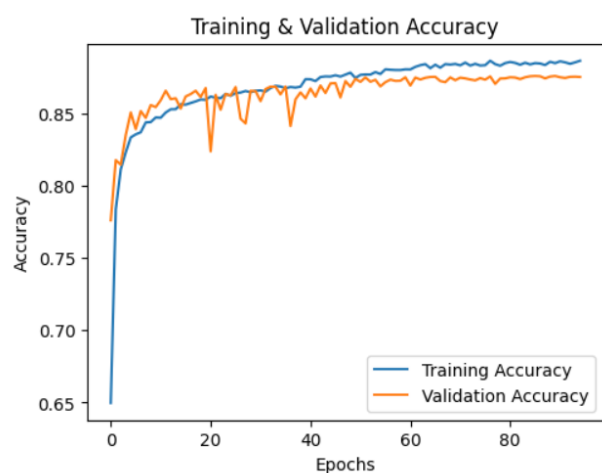
Trainable params: 6,209 (24.25 KB)

Non-trainable params: 0 (0.00 B)

3.2 Training results

At final epoch, the model achieved a train accuracy of 88.54% and train loss of 0.2728, and a validation accuracy of 87.51% and validation loss of 0.2749. From the graphs, validation accuracy and loss were very similar to train accuracy and loss, which shows that the trained model is not overfitting or underfitting.

```
Epoch 11/1000
1505/1505 8s 5ms/step - accuracy: 0.8482 - loss: 0.3429 - val_accuracy: 0.8590 - val_loss: 0.3180 - learning_rate: 0.0010
Epoch 12/1000
1505/1505 8s 5ms/step - accuracy: 0.8498 - loss: 0.3423 - val_accuracy: 0.8656 - val_loss: 0.3170 - learning_rate: 0.0010
...
Epoch 95/1000
1496/1505 0s 5ms/step - accuracy: 0.8854 - loss: 0.2728
Epoch 95: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.
1505/1505 8s 5ms/step - accuracy: 0.8854 - loss: 0.2728 - val_accuracy: 0.8751 - val_loss: 0.2749 - learning_rate: 1.5625e-05
```



3.3 Evaluating the model

Evaluating the model on the test dataset yielded an accuracy of 87.01% and a loss of 0.2834. From the confusion matrix, the misclassification rate for both 'female' and 'male' labels are similar.

```
# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_mfccs, test_labels)

predictions = model.predict(test_mfccs)

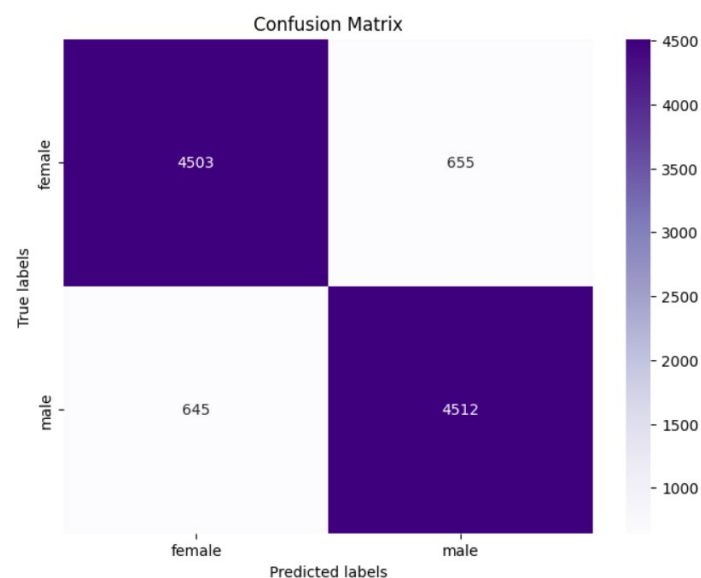
# Convert the labels back to their original values
test_labels_categ = label_encoder.inverse_transform(test_labels)

# Convert the predictions to binary values
predicted_labels = (predictions > 0.5).astype(int) # Converts probabilities to 0 or 1
predicted_labels = label_encoder.inverse_transform(predicted_labels)

# Create confusion matrix
conf_matrix = confusion_matrix(test_labels_categ, predicted_labels)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_, cmap='Purples')
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix')
plt.show()
```

```
323/323 ————— 1s 2ms/step - accuracy: 0.8701 - loss: 0.2834
323/323 ————— 1s 1ms/step
```



4. TensorFlowLite Model Conversion

4.1 Converting to TFLite model

The model was converted to the TFLite model with int8 weight and bias quantization, and activation value quantization. The resultant size was 14312 bytes.

4.2 Evaluating the TFLite model

Testing the TFLite model with the test dataset yielded an accuracy of 87.46%, which is similar to the unquantized test accuracy of 87.01%.

```
# Show accuracy
model_quantized_predictions = np.array(model_quantized_predictions)
model_quantized_predicted_labels = (model_quantized_predictions > 0.5).astype(int).flatten()
model_quantized_predicted_labels = label_encoder.inverse_transform(model_quantized_predicted_labels)

# Calculate accuracy
accuracy = np.mean(model_quantized_predicted_labels == test_labels_cat)
print(f"Quantized model accuracy: {accuracy * 100:.2f}%")
```

Quantized model accuracy: 87.46%



4.3 Converting to C header file

The model was then converted to a C header file for use on the Arduino.

5. Arduino Implementation

5.1 Setting the parameters

The same MFCC parameters were used as in the Python implementation, and an audio buffer to hold 3184ms of audio was created.

5.2 Setting up the PDM

The PDM callback was set up to read data into the buffer only if it is empty. If the buffer is full, the *isBufferFull* flag will be set to indicate that the buffer is full and ready for processing.

The callback function will not store any new data to the buffer as long as the flag is set to avoid overwriting the current data before it has been processed.

```
void onPDMdata()
{
    /* Callback function to handle PDM data. */

    int bytesAvailable = PDM.available();
    if (bytesAvailable > 0)
    {
        int16_t pdmBuffer[bytesAvailable / 2];
        int samplesRead = PDM.read(pdmBuffer, bytesAvailable) / 2;

        for (int i = 0; i < samplesRead; i++)
        {
            if (!isBufferFull)
            {
                audio_buffer[bufferIndex] = pdmBuffer[i];
                bufferIndex++;

                // If the buffer is full,
                if (bufferIndex >= BUFFER_SIZE)
                {
                    isBufferFull = true; // indicate ready for processing
                    bufferIndex = 0;     // reset for next round
                }
            }
        }
    }
}
```

5.3 Overall flow

Thus, with the PDM callback set up, the Arduino will constantly capture audio and store it to the audio buffer. Once the buffer is full, the *isUsingInputTensor* flag will be set and the data in the buffer will be processed and used for inference.

Once inference is completed, the *isUsingInputTensor* flag will be unset to allow for processing of the next round of data.

```
void loop()
{
    if (isBufferFull && !isUsingInputTensor)
    {
        isUsingInputTensor = true;
        processBuffer();
        isUsingInputTensor = false;
    }
}
```

5.4 Processing the buffer

When the audio buffer is full, the data in the buffer will be processed. The whole buffer is first normalized to $[-1, 1]$, and then the MFCCs for each frame are calculated and stored directly to the input tensor.

Before running inference, the *isBufferFull* flag will be unset to indicate that the audio buffer can proceed to store the next round of data.

```
void processBuffer()
{
    /* Process the audio buffer. */

    Serial.println("Processing buffer.");

    // Find the max value in the audio buffer
    int16_t maxVal = 0;
    for (int i = 0; i < BUFFER_SIZE; i++)
    {
        int16_t absValue = abs(audio_buffer[i]);
        if (absValue > maxVal)
        {
            maxVal = absValue; // Ensure maxVal is the largest absolute value
        }
    }

    // Process and save frames to input tensor
    for (int i = 0; i < NUM_FRAMES; i++)
    {
        // Create a frame from the audio buffer
        float frame[FRAME_SIZE];
        for (int j = 0; j < FRAME_SIZE; j++)
        {
            frame[j] = (float)audio_buffer[i * HOP_LENGTH + j];
        }

        if (maxVal > 0)
        {
            // Normalize the audio buffer
            for (int n = 0; n < FRAME_SIZE; n++)
            {
                frame[n] = frame[n] / (float)maxVal;
            }
        }

        // Compute MFCCs for the frame
        float mfcc[NUM_MFCC];
        computeMFCC(frame, mfcc, FRAME_SIZE);

        // Copy the MFCCs to the input tensor with offset for each frame
        for (int k = 0; k < NUM_MFCC; k++)
        {
            // Input tensor is of shape: [1, 13, 500, 1]
            // Flatbuffer: [MFCC0_frame0, MFCC0_frame1, ..., MFCC0_frame499, MFCC1_frame0, MFCC1_frame1, ..., MFCC1_frame499, ...]
            // Copy the MFCC to the correct location for each frame, using frame index of frameCount
            input->data.f[k * NUM_FRAMES + i] = mfcc[k]; // i is the frame index
        }
    }

    // Audio buffer already copied to input tensor
    isBufferFull = false;

    // Run inference
    runInference();
}
```

5.5 Computing MFCCs

The process to compute MFCCs on the Arduino is aligned with the steps used by 'librosa' in Python. For each frame, a Hanning window is applied, then FFT is performed, then the power spectrum is computed, then the Mel filter bank is applied to the power spectrum, then the Mel spectrum is converted to the log scale, then DCT is performed, and finally orthonormal normalization is applied. These are the exact same steps used in the 'librosa' method.

```
void computeMFCC(float *frame, float *mfcc, int frameSize)
{
    /* Compute MFCCs for a single frame.*/

    // Step 1: Apply Hanning window
    for (int i = 0; i < frameSize; i++)
    {
        frame[i] *= 0.5f * (1 - cos(2 * M_PI * i / (frameSize - 1))); // Hanning window

        real[i] = frame[i]; // Copy audio data to real part
        imag[i] = 0.0f;      // Initialize imaginary part to zero
    }

    // Step 2: Perform FFT
    FFT.compute(real, imag, frameSize, FFT_FORWARD);

    // Step 3: Compute power spectrum (square of magnitudes)
    float powerSpectrum[frameSize / 2];
    for (int i = 0; i < frameSize / 2; i++)
    {
        powerSpectrum[i] = real[i] * real[i] + imag[i] * imag[i];
    }

    // Step 4: Apply Mel filter bank
    float melSpectrum[NUM_MEL_FILTERS] = {0.0f};
    for (int filter = 0; filter < NUM_MEL_FILTERS; filter++)
    {
        for (int bin = 0; bin <= frameSize / 2; bin++)
        {
            melSpectrum[filter] += mel_filter_bank[filter][bin] * powerSpectrum[bin];
        }
    }

    // Step 4.5: Convert to log scale
    for (int filter = 0; filter < NUM_MEL_FILTERS; filter++)
    {
        if (melSpectrum[filter] != 0)
        {
            melSpectrum[filter] = 10 * log10(melSpectrum[filter]);
        }
    }

    // Step 5: Compute DCT
    for (int i = 0; i < NUM_MFCC; i++)
    {
        mfcc[i] = 0.0f;
        for (int j = 0; j < NUM_MEL_FILTERS; j++)
        {
            mfcc[i] += melSpectrum[j] * cos(M_PI * i * (2 * j + 1) / (2 * NUM_MEL_FILTERS));
        }
    }

    // Step 6: Apply orthonormal normalization
    mfcc[0] *= sqrt(1.0f / NUM_MEL_FILTERS); // scale the first coefficient
    for (int i = 1; i < NUM_MFCC; i++)
    {
        mfcc[i] *= sqrt(2.0f / NUM_MEL_FILTERS);
    }
}
```

5.6 Running inference

The model is then invoked to produce the output. Since the 'female' label was encoded to 0 and 'male' to 1 during training, a score of <0.5 indicates a 'female' classification and a 'male' classification otherwise.

```
void runInference()
{
    /* Run inference on the input tensor. */
    Serial.println("Running inference.");

    TfLiteStatus invokeStatus = interpreter->Invoke();
    if (invokeStatus != kTfLiteOk)
    {
        MicroPrintf("Invoke failed");
        return;
    }

    float score = output->data.f[0]; // Use the single score from the output
    float confidence;

    // Classify based on the score (sigmoid output)
    Serial.print("Predicted gender: ");
    if (score < 0.5) // If the score is < 0.5, classify as Female
    {
        Serial.println("Female");
        confidence = (1.0f - score) * 100;
    }
    else // If the score is > 0.5, classify as Male
    {
        Serial.println("Male");
        confidence = score * 100;
    }

    // Print the score
    Serial.print("Confidence: ");
    Serial.print(confidence, 2);
    Serial.println("%");
    Serial.println("");

    // Clear or reset the input tensor
    memset(input->data.f, 0, input->bytes); // set all bytes of the input tensor to zero
}
```